



Tema 17 - 20

1 DAW

**Faisal Alahdab, Marcos Rodriguez,
Alejandro Gomez Bermudo**

[Enlace GitHub:](#)



1. Cartelera.java:

La clase **Cartelera** funciona como la parte principal que conecta los distintos elementos del sistema, ya que relaciona una **película** con una **sala** y además guarda información propia de cada sesión, como la hora de proyección y la puntuación.

Las mejoras que se han aplicado:

1. Mejora en la salida por consola con printf

En vez de mostrar la información usando concatenaciones simples de texto, se ha utilizado (`System.out.printf`) para que la salida quede más ordenada y fácil de leer.

Por qué se hizo esto?

Porque al usar formatos como `%-25s`, se reserva un espacio fijo para el título de la película, lo que hace que el resto de datos (sala, horario y nota) aparezcan alineados correctamente. Así la información en consola se ve mucho más limpia y profesional.

2. Mejor organización mediante composición de objetos

En lugar de guardar solo el nombre de la película como un `String`, la clase trabaja directamente con un objeto de tipo `Pelicula`.

Ventaja:

Esto hace que el diseño sea más flexible, porque si más adelante la clase `Pelicula` incluye nuevos datos como director, duración o género, se podrán usar fácilmente sin tener que cambiar la estructura de `Cartelera`.



3. Uso del encapsulamiento para proteger los datos

Todos los atributos de la clase (película, puntuación, sala y horario) se han declarado como privados. Porque de esta forma, no se pueden modificar directamente desde otras clases, sino que el acceso se controla mediante métodos getter. Esto ayuda a mantener la seguridad de los datos y evita cambios accidentales en la información de cada sesión.

4. Documentación con JavaDoc

También se añadieron comentarios con formato JavaDoc para explicar el funcionamiento de la clase y de sus métodos.

Para qué sirve esto?

Para que el código sea más fácil de entender para otros programadores o incluso para uno mismo en el futuro.

2. Main.java:

La clase Main ha pasado de tener todo el código amontonado a ser un "director de orquesta". Ahora no contiene la lógica de cómo se hace una reserva o cómo se crea una sala, sino que simplemente llama a los métodos adecuados cuando toca.

Las mejoras que se han aplicado:

1. Modularización completa del código He sacado bloques enormes de código fuera del main y los he repartido en métodos como `inicializarSalas()`, `inicializarCartelera()` o `procesoDeReserva()`.

¿Por qué se hizo esto? Porque antes el main era larguísimo y difícil de leer.

Ahora, al abrir el archivo, ves un menú muy limpio y sabes exactamente qué hace cada parte sin perderte en los detalles técnicos.



2. Implementación de una lectura de datos segura (leerEntero) He creado un método específico para leer los números que introduce el usuario.

Ventaja: Este método evita que el programa "pete" si el usuario escribe una letra por error en vez de un número. El programa ahora detecta el error, limpia el fallo y te pide el dato de nuevo, lo que lo hace mucho más robusto.

3. Mejora en la experiencia de usuario (Sincronización de índices) Hemos programado el sistema para que el usuario siempre trabaje con números naturales (del 1 al 5).

Cómo funciona? El usuario elige la "Película 1" o la "Fila 1", y el programa resta un -1 internamente para que coincida con el índice 0 de los arrays de Java. Así evitamos que el usuario tenga que saber informática para usar el programa.

4. Bucle de reserva múltiple He añadido un bucle do -while dentro del proceso de reserva.

Para qué sirve? Antes, para comprar tres entradas de la misma película, tenías que volver al menú principal tres veces. Ahora, una vez que eliges la película, el programa te pregunta si quieres añadir otro asiento, ahorrando mucho tiempo al usuario.

5. Validación de rangos y errores Se han incluido comprobaciones en cada paso

Justificación: Esto impide que el programa intente acceder a una posición del array que no existe (evitando el famoso error `ArrayIndexOutOfBoundsException`), haciendo que el sistema sea a prueba de fallos

6. Gestión de memoria y cierre de recursos Al final del programa he añadido `sc.close()`.

Por qué? Es una buena práctica esencial para cerrar el flujo de entrada de datos (Scanner) y liberar la memoria que estaba usando el programa antes de cerrarse



3. Pelicula.java:

La clase Pelicula es el modelo básico de datos del programa. Se ha diseñado para ser un objeto "limpio" que solo transporte la información necesaria de cada cinta.

Las mejoras que se han aplicado:

1. Encapsulamiento de los atributos He definido titulo, duracion y genero como variables privadas (private).

Por qué se hizo esto? Para que nadie pueda modificar el nombre de una película o su género desde fuera de la clase por error. Es la forma correcta de programar en Java para que los datos estén protegidos y el programa sea más estable.

2. Constructor específico Se ha creado un constructor que recibe los tres parámetros fundamentales al mismo tiempo.

Ventaja: Esto me permite crear películas rápidamente en el Main en una sola línea de código, asegurando que ninguna película se quede "vacía" o sin datos importantes al empezar el programa.

3. Diseño flexible para el futuro Aunque ahora el programa solo usa principalmente el título para las reservas, la clase ya está preparada con los campos de duracion y genero.

Justificación: Si más adelante el profesor pide que el ticket muestre cuánto dura la película o de qué tipo es, no tengo que modificar la estructura del resto del programa; los datos ya están ahí guardados en el objeto desde el principio.



4. Simplificación de métodos de acceso Solo he incluido el método `getTitulo()` porque es el que realmente necesita la clase `Cartelera` y el `Main` para mostrar los menús.

Por qué? Para no llenar el código de métodos que no se usan. Si en el futuro necesito la duración, solo tengo que añadir su getter, manteniendo el código actual lo más corto y legible posible.

4. Sala.java:

Esta clase es la responsable de gestionar el estado físico del cine. Se ha refactorizado para que la visualización y la reserva de asientos sean mucho más eficientes y claras para el usuario.

Las mejoras que se han aplicado:

1. Representación visual mediante matriz booleana Se ha utilizado una matriz `boolean[][]` de 5x5 para representar los asientos.

Por qué se hizo así? Es la forma más eficiente de controlar si un sitio está libre o no. Un valor `false` significa que el asiento está disponible y `true` que ya está reservado. Esto ocupa el mínimo espacio en memoria y permite consultas instantáneas.

2. Simplificación del dibujo con el operador ternario En el método `mostrarAsientos()`, se ha sustituido una estructura `if-else` larga por un operador ternario: `asientos[i][j] ? "[ X ]" : ....`

Ventaja: El código queda mucho más limpio y profesional. Además, permite que el mapa de la sala se dibuje dinámicamente: si el asiento está ocupado muestra una "X", y si no, muestra sus propias coordenadas, lo que ayuda al usuario a elegir su sitio sin equivocarse.



3. Sincronización visual (Interfaz vs Lógica) Aunque internamente la matriz empieza en 0, en el método `mostrarAsientos()` se imprime sumando +1 a los índices.

¿Para qué sirve? Para que el usuario vea en pantalla "Fila 1" o "Asiento 1,1" en lugar de "Fila 0". De esta forma, lo que el usuario ve coincide exactamente con lo que el programa le pide introducir, evitando confusiones de "lógica de programador".

4. Validación de reserva mediante retorno booleano El método `reservarAsiento` no solo cambia el estado del asiento, sino que informa del éxito de la operación devolviendo `true` o `false`.

Justificación: Esto permite separar la lógica de la sala de la interfaz del usuario. La sala solo se encarga de intentar la reserva; si no puede (porque ya está ocupado), avisa al Main para que este decida qué mensaje de error mostrar.

5. Encapsulamiento y Control de Acceso La matriz de asientos es privada.

Seguridad: Esto garantiza que la única forma de "sentar" a alguien en la sala es pasando por el método `reservarAsiento`, el cual podría incluir en el futuro reglas adicionales (como no dejar asientos libres entre personas) sin romper el resto del código.